



Universidad
Nacional
de Loja



Carrera de Ingeniería en
Sistemas / Computación

UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables ·
Ingeniería en Ciencias de la Computación

Evaluación del rendimiento de un clúster de bajo costo basado en Raspberry Pi orquestado con Kubernetes

Autor

Diego Oswaldo Márquez Paccha

Director

Ing. Cristian Ramiro Narváez Guillén, Mg. Sc.

Loja – Ecuador 2026

Agenda

- Problemática y pregunta de investigación
- Objetivos
- Objetivo 1 — Construcción del prototipo físico
- Objetivo 2 — Evaluación del rendimiento
- Discusión de resultados
- Conclusiones y recomendaciones

Problemática y Pregunta

CONTEXTO Depender de la nube comercial y de servidores tradicionales es una **barrera de costo** para universidades y entornos con pocos recursos.

ALTERNATIVA Un **clúster de Raspberry Pi** con una versión ligera de Kubernetes: económico y reproducible.

PREGUNTA DE INVESTIGACIÓN ¿Cuál es el **tiempo de respuesta** y la **cantidad de peticiones por segundo** de un clúster con **Kubernetes** para aplicaciones web, al someterlo a una **carga** en una arquitectura de **bajo costo**?

LIMITACIÓN Equipos de poca capacidad, sobre todo en **red**: en la Raspberry Pi 3 la red pasa por el **puerto USB**.

VACÍO Poca evidencia real de su **rendimiento bajo carga** y de **qué lo limita**.

Conceptos clave

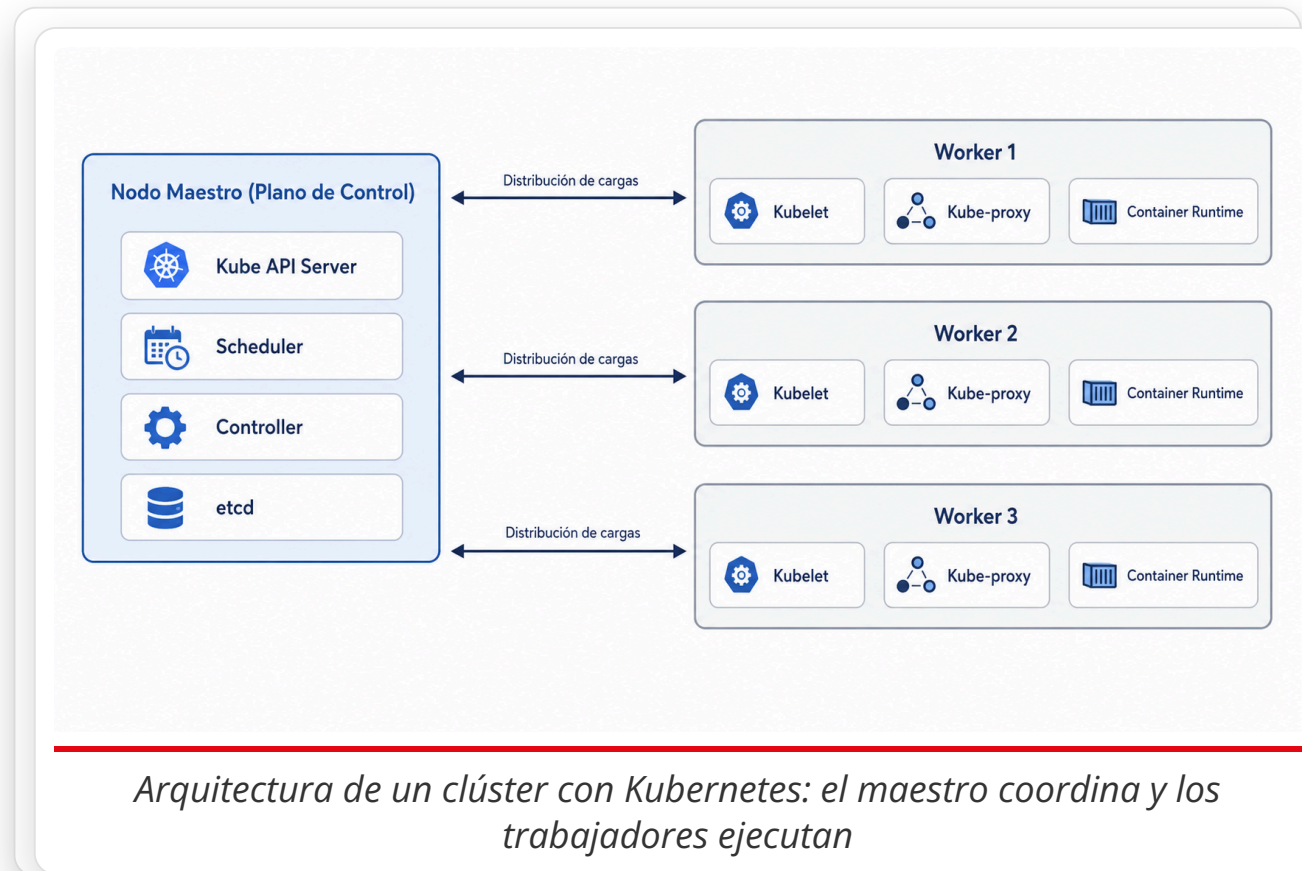
Clúster — varias computadoras que trabajan juntas como si fueran una sola.

Kubernetes y K3s — coordinan las aplicaciones; K3s es la versión ligera para equipos pequeños.

Maestro y trabajadores — el maestro coordina; los trabajadores ejecutan las aplicaciones.

Latencia y throughput — el tiempo de respuesta y las peticiones por segundo: las métricas del estudio.

Freno por calor — a 80–85 °C el equipo baja la velocidad del procesador para protegerse.



Objetivos

Objetivo General

Medir el nivel de **latencia** y **throughput** que presenta un clúster orquestado con Kubernetes para el despliegue de aplicaciones web contenerizadas, al someterse a una simulación de carga en una arquitectura de bajo costo.

Objetivos Específicos

1. **Construir el prototipo físico** del clúster de bajo costo, abarcando desde el diseño de la topología de red hasta la instalación y configuración base del sistema operativo en los nodos, para garantizar una base estable y funcional al momento de la orquestación con Kubernetes.
2. **Orquestar el despliegue de aplicaciones web** en el clúster mediante Kubernetes, sometiendo el sistema a simulaciones de carga para registrar las métricas de latencia y throughput.

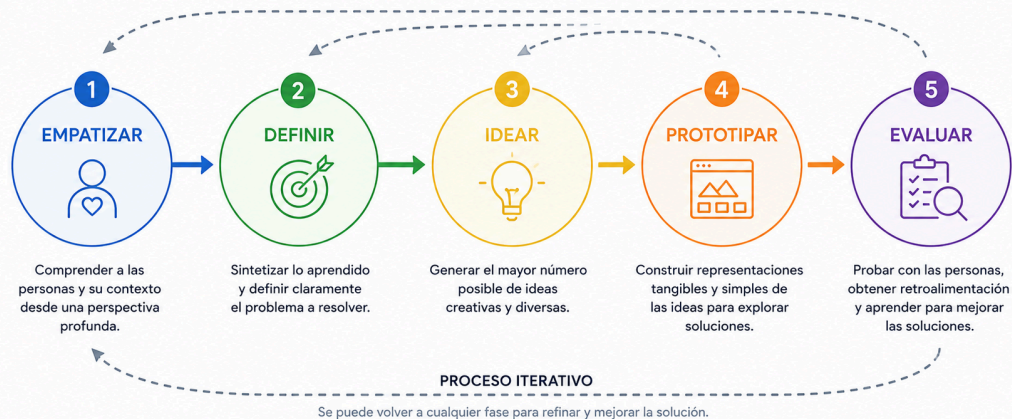
Objetivo 1 — Construcción del prototipo físico

01

Metodología — Diseño de la carcasa

Las cinco fases del proceso de Design Thinking

d.school de Stanford

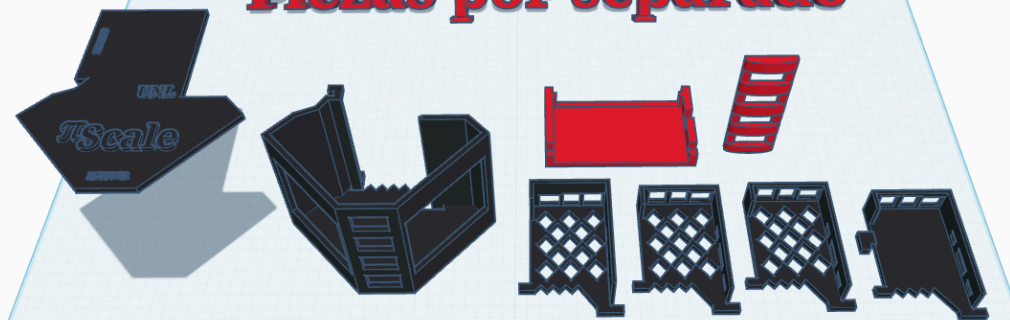


Método Design Thinking: 3 versiones hasta el diseño final

Ensamblaje virtual



Piezas por separado

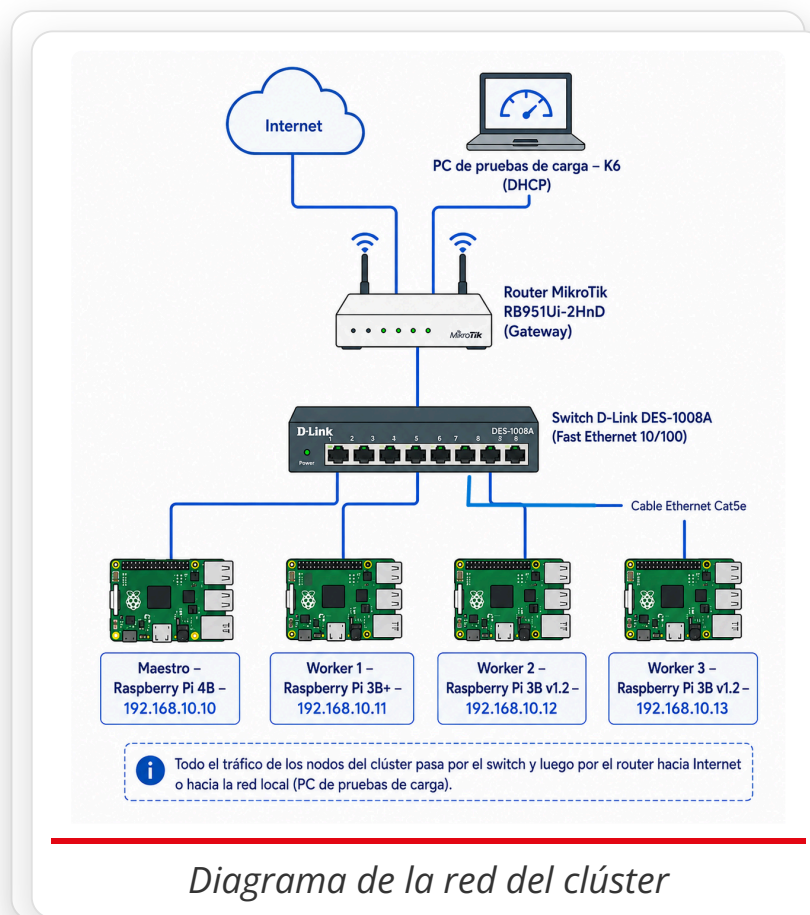


Plano de trabajo

Diseño en 3D de la carcasa (con enfriamiento y red integrada)

Metodología — Equipos y red

- ▶ **1 nodo maestro** (Raspberry Pi 4) que coordina
- ▶ **3 trabajadores** (Raspberry Pi 3) que ejecutan
- ▶ Conmutador y enrutador de red
- ▶ **Direcciones de red fijas** para cada nodo



Metodología — Preparación y orquestación

- ▶ **Sistema estable:** se **bloquearon las versiones** del núcleo (kernel) y el firmware, y se desactivaron las actualizaciones automáticas → un entorno que no cambia entre pruebas.
- ▶ **Orquestación con K3s:** primero el nodo maestro, luego cada trabajador se une al clúster.
- ▶ **Pantallas por nodo:** un programa en Python muestra en vivo el procesador, la temperatura y la memoria.



Todos los comandos

Escanea el código o visita:

github.com/eval-cluster/cluster

```
1 # Bloquear versiones del núcleo y firmware
2 sudo apt-mark hold linux-image-raspi \
3   linux-headers-raspi linux-raspi linux-firmware
4
5 # Instalar K3s en el nodo maestro (sin Traefik)
6 curl -sfL https://get.k3s.io | sh -s - --disable traefik
7
8 # Unir un trabajador al clúster
9 curl -sfL https://get.k3s.io | \
10   K3S_URL=https://192.168.10.10:6443 \
11   K3S_TOKEN=<token> sh -
```

Resultado — Prototipo físico

- ▶ Carcasa **impresa en 3D**: 4 nodos, ventiladores y conmutador en un solo cuerpo
- ▶ Una **pantalla por nodo** con su estado en vivo
- ▶ **Bajo costo**: ~464 dólares → hasta **87 % de ahorro** frente a la nube



Diseño 3D del clúster

Escanea el código o visita:

tinkercad.com/things/cluster-4-raspberrypi



Clúster ensamblado



Pantallas con el estado de cada nodo

Resultado — Clúster operativo

- ▶ Los **4 nodos activos** y reconocidos por Kubernetes (K3s)
- ▶ **Ningún fallo** de nodo en las 21 ejecuciones
→ un entorno **estable y determinista**

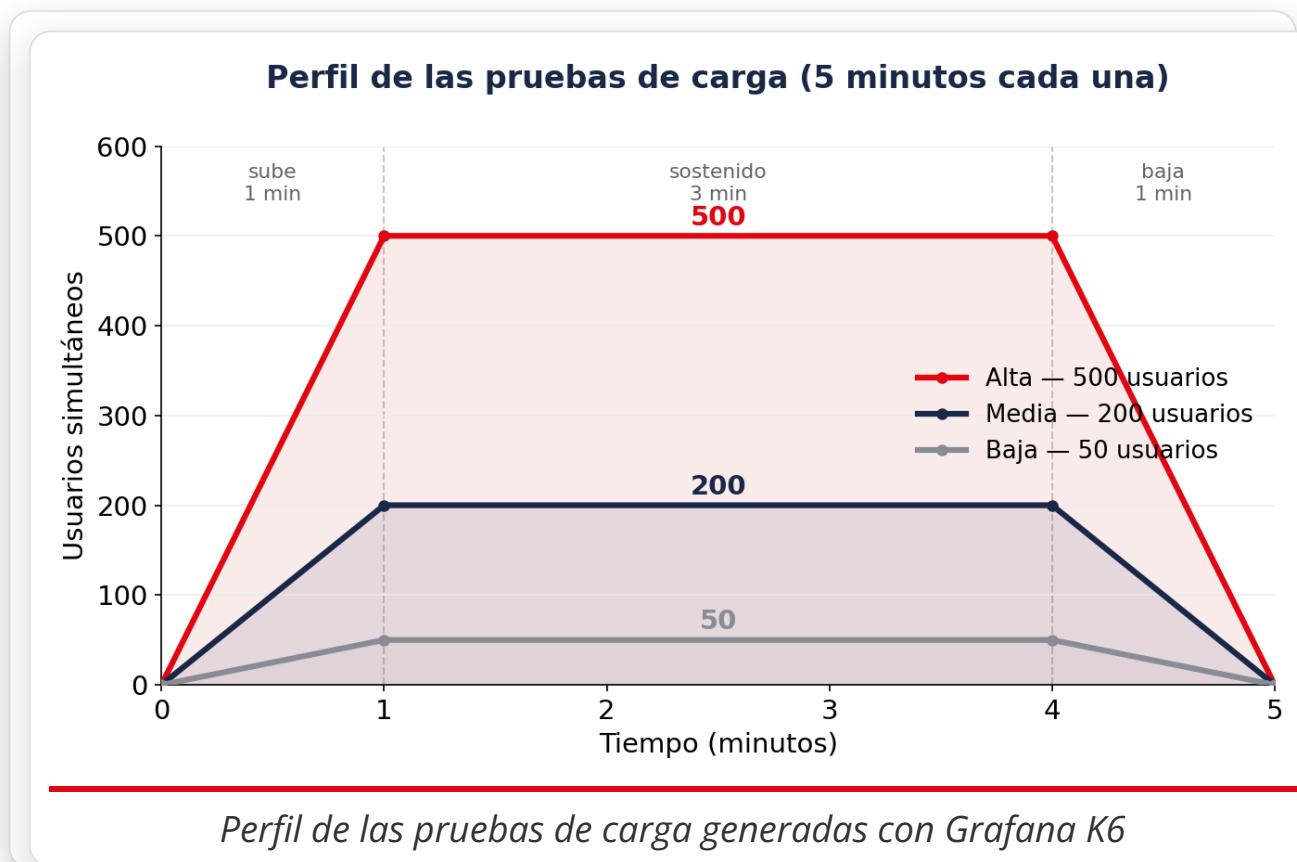
```
cluster@master:~$ kubectl get nodes -o wide
NAME      STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
master    Ready     control-plane  2d22h   v1.35.5+k3s1  192.168.10.10   <none>         Ubuntu 24.04.4 LTS   6.8.0-1057-raspi  containerd://2.2.3-k3s1
worker1   Ready     <none>     2d22h   v1.35.5+k3s1  192.168.10.11   <none>         Ubuntu 24.04.4 LTS   6.8.0-1057-raspi  containerd://2.2.3-k3s1
worker2   Ready     <none>     2d22h   v1.35.5+k3s1  192.168.10.12   <none>         Ubuntu 24.04.4 LTS   6.8.0-1057-raspi  containerd://2.2.3-k3s1
worker3   Ready     <none>     2d22h   v1.35.5+k3s1  192.168.10.13   <none>         Ubuntu 24.04.4 LTS   6.8.0-1057-raspi  containerd://2.2.3-k3s1
cluster@master:~$
```

Los cuatro nodos activos (kubectl get nodes)

| Objetivo 2 — Evaluación del rendimiento

Metodología — Cómo se midió

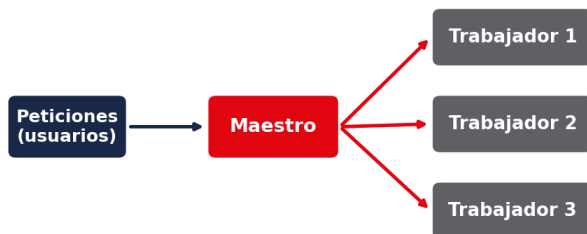
- ▶ **Dos escenarios:** con y sin ventiladores.
Aplicación de prueba: el servidor web **NGINX**.
- ▶ **Tres niveles de carga:** 50, 200 y 500 usuarios simultáneos (uso normal, intermedio y exigencia máxima).
- ▶ **Monitoreo** del procesador, la red y la temperatura con **Prometheus y Grafana**.



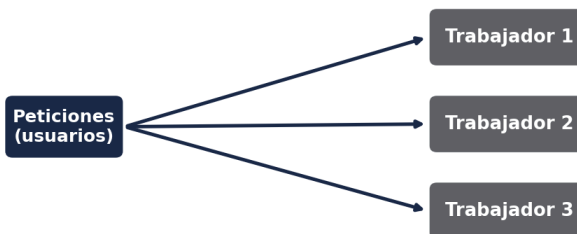
Metodología — Hallar el cuello de botella

Experimento: ¿por dónde entran las peticiones?

Centralizado (original)



Repartido (experimento)



Se cambia solo por dónde entran las peticiones

Experimento extra: en lugar de enviar todo al maestro, se **reparten** las peticiones entre los trabajadores.

Si repartir **mejora mucho** → el problema era la concentración en el maestro.

Si **no mejora** → el problema está en la **red de los trabajadores**.

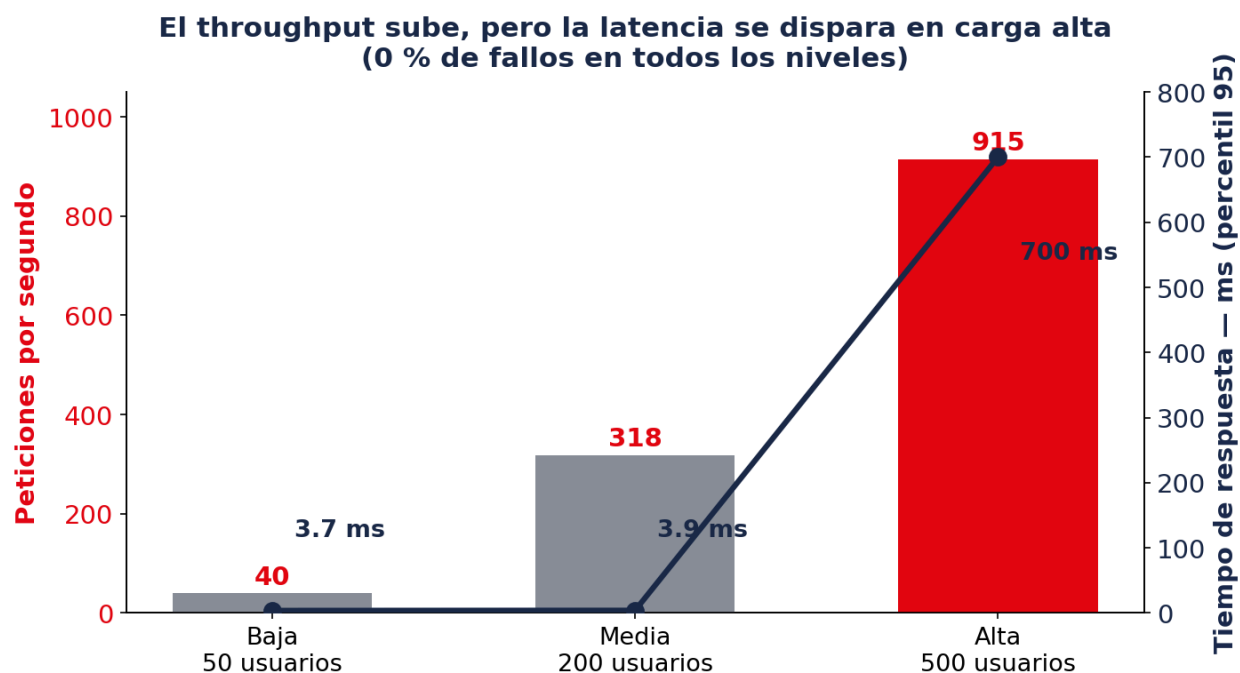
Resultado — Despliegue de la aplicación web

- ▶ Se desplegó el servidor web **NGINX** con **3 réplicas**, repartidas por Kubernetes en los **3 nodos trabajadores**.
- ▶ Confirma que **K3s orquesta** correctamente sobre hardware embebido, incluso en nodos de solo **1 GB de RAM**.

```
cluster@master: ~  
cluster@master:~$ kubectl get pods -o wide  
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES  
nginx-deployment-7df4666576-6nzcx   1/1     Running   0          2d12h  10.42.4.3     worker3   <none>            <none>  
nginx-deployment-7df4666576-hfgsq   1/1     Running   0          2d12h  10.42.3.3     worker2   <none>            <none>  
nginx-deployment-7df4666576-p885g   1/1     Running   0          2d12h  10.42.1.3     worker1   <none>            <none>  
cluster@master:~$
```

Una réplica de NGINX en cada trabajador (kubectl get pods -o wide)

Resultado — Rendimiento por carga



Throughput (barras) y latencia (línea) según la carga

- ▶ El **throughput** crece con la carga: de 40 a **915** peticiones por segundo.
- ▶ La **latencia** se mantiene baja hasta 200 usuarios y **se dispara** a 700 ms en carga alta.

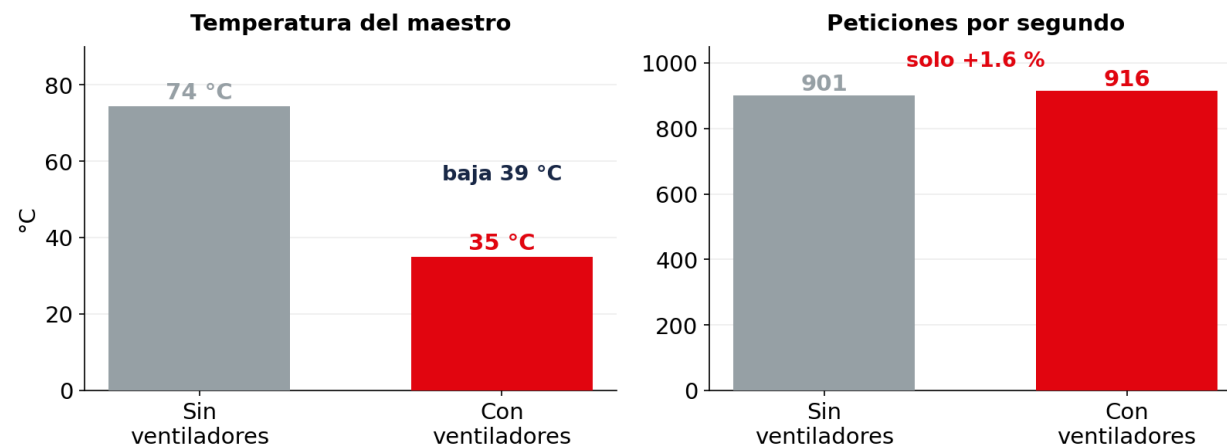
Responde bien hasta 200 usuarios y **se satura en 500** — más lento, pero **sin un solo fallo**.

Resultado — Enfriar casi no mejora

- ▶ Los ventiladores bajaron la temperatura del maestro de **74 °C a 35 °C**.
- ▶ Pero el rendimiento **apenas cambió**: solo **1.6 %** más peticiones por segundo.

La temperatura **no limita** el rendimiento web, porque el procesador **nunca trabaja al límite**.

Bajó mucho la temperatura, pero el rendimiento casi no cambió

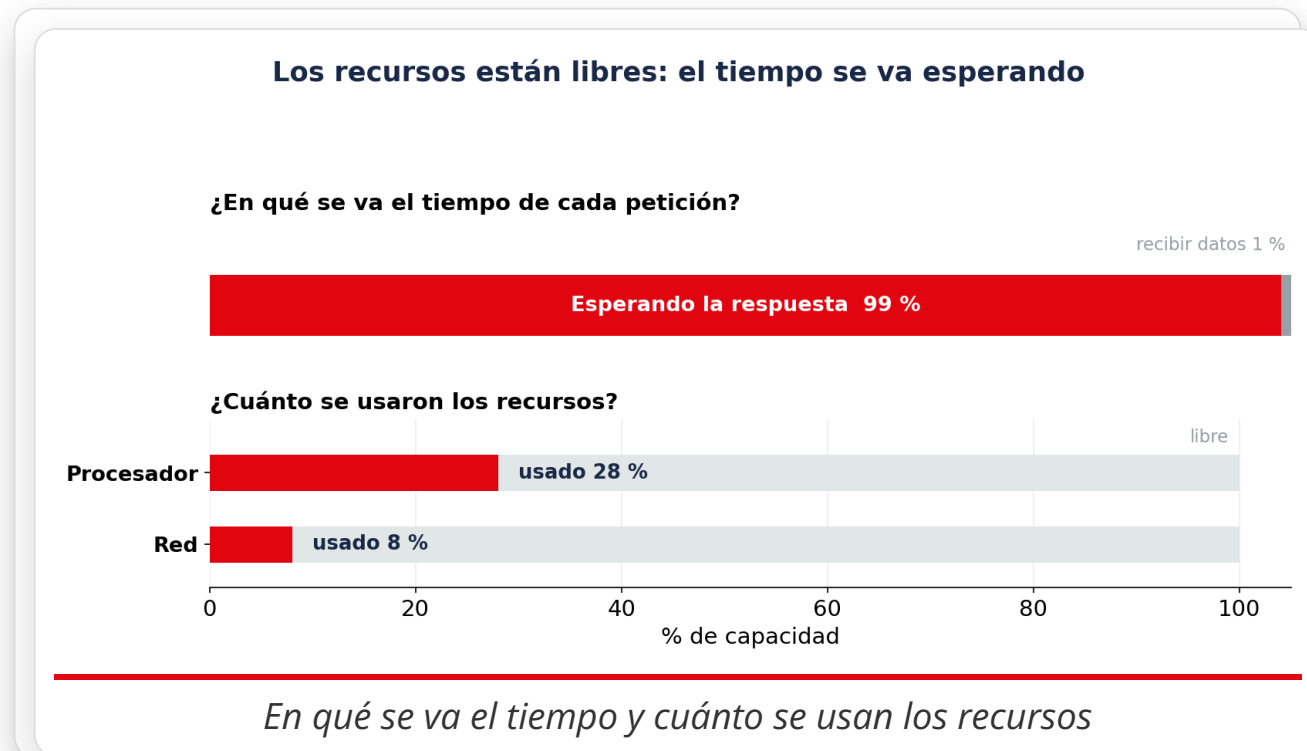


Mucho enfriamiento, casi ninguna mejora

Resultado — Qué está pasando por dentro

- ▶ El **procesador** quedó casi libre (menos del 28 %)
- ▶ La **red** se usó solo al 8 %
- ▶ Casi todo el tiempo (**99 %**) se va **esperando la respuesta**, no enviando datos

Ni el procesador ni la red están saturados: el problema **no es la potencia, sino la espera**.

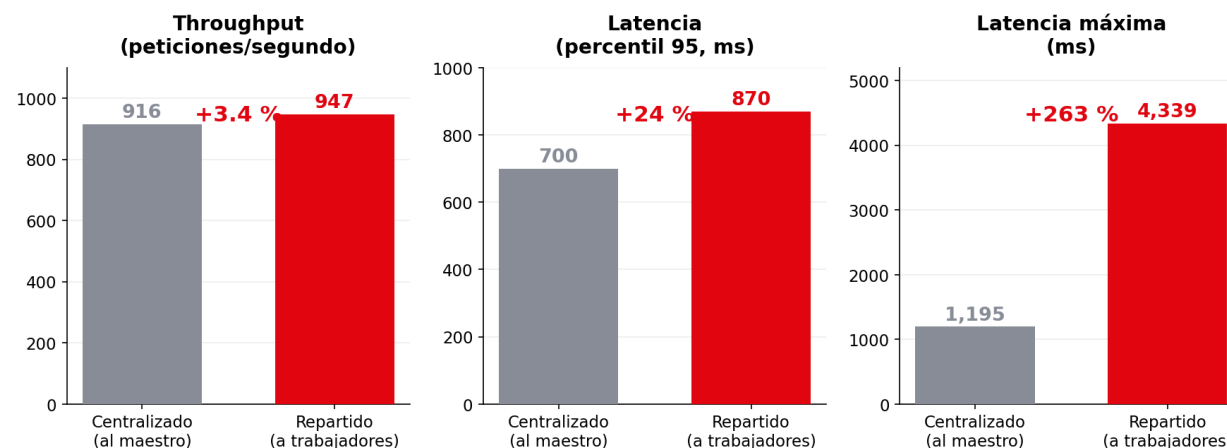


Resultado — Repartir la carga desde la PC

- ▶ Desde la PC de pruebas se **repartió la carga** directamente entre los 3 trabajadores, en vez de enviarla toda al maestro.

No era la concentración en el maestro: el throughput apenas subió (**3.4 %**) y la respuesta se volvió **más irregular**. Cada trabajador recibe el tráfico por su **propia conexión (puerto USB)**, igual de limitada.

Repartir la carga casi no sube el rendimiento, pero empeora la consistencia



Centralizado frente a repartido, con carga alta

Resultado — El cuello de botella

¿Qué limita el rendimiento? Se descartaron las causas



Temperatura

nunca hubo freno por calor



Procesador

usado menos del 28 %



Ancho de banda

solo el 8 % de la red



Concentración en el maestro

repartir dio apenas +3.4 %



Red de los trabajadores

pasa por el puerto USB

◀ CAUSA

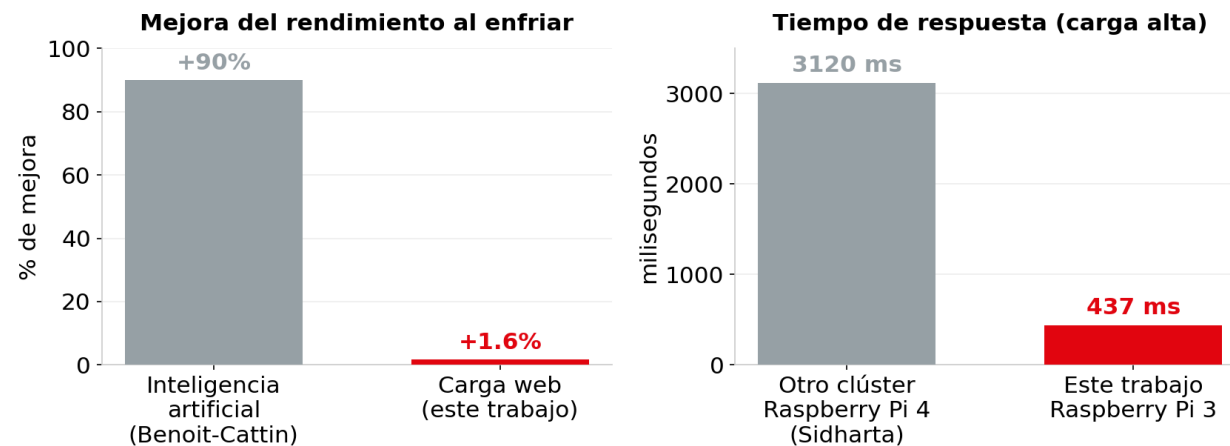
Descarte de causas: el límite es la red de los trabajadores

| **Discusión**

Discusión — Comparación con otros estudios

- ▶ **Enfriamiento (Benoit-Cattin y otros):**
lograron gran mejora, pero con **inteligencia artificial**, que exige mucho al procesador. Una carga web no lo hace.
- ▶ **Tiempo de respuesta (Sidharta y otros):**
con Raspberry Pi 4 y red rápida tuvieron más demora que nosotros → la **configuración** pesa tanto como el equipo.
- ▶ **Costo y fiabilidad:** ~464 dólares (en línea con Shoop y Sankiti) y **ningún fallo**, frente a las fallas de Lakshminarayanan.

¿Por qué estos resultados difieren de otros estudios?



Contraste de este trabajo frente a otros estudios

El **freno por calor solo aparece con tareas que exigen mucho al procesador**. Una carga web no lo hace: por eso enfriar no mejora el rendimiento.

Discusión — Qué significa

- ▶ Para cargas web, conviene invertir primero en **mejor red**, no en enfriamiento.
- ▶ El enfriamiento igual sirve: **protege los equipos** (más vida útil) y los prepara para **tareas más exigentes** en el futuro.
- ▶ Con esto queda claro **qué mejorar primero** en un clúster de este tipo.

Discusión — Hasta dónde llega este trabajo

- ▶ **Repeticiones:** 3 por prueba → tendencias claras y consistentes, pero **no un análisis estadístico formal**.
- ▶ **Un solo tipo de carga:** páginas web sencillas, que no exigen al procesador. Otras cargas (bases de datos, inteligencia artificial, archivos grandes) podrían tener **otro factor limitante** e incluso activar el freno por calor.
- ▶ **Equipos distintos entre sí:** un maestro Raspberry Pi 4 y tres trabajadores Raspberry Pi 3 de dos versiones → refleja un caso real, pero **dificulta aislar cada variable**.
- ▶ **Sin grupo de comparación:** no se probó un clúster con red dedicada; la conclusión sobre la red se apoya en el **descarte de causas y en la literatura**, no en una comparación directa.
- ▶ **Pruebas cortas (5 minutos):** miden el rendimiento estable, pero **no el desgaste** de un uso prolongado (por ejemplo, las tarjetas de memoria).

| Conclusiones y Recomendaciones

Respuesta a la pregunta de investigación

*¿Cuál es el nivel de **latencia** y **throughput** que presenta un clúster orquestado con Kubernetes para el despliegue de aplicaciones web contenerizadas, al someterse a una simulación de carga en una arquitectura de bajo costo?*

THROUGHPUT

≈ **915**

peticiones por segundo, bajo carga alta
(40 y 318 en carga baja y media)

LATENCIA

≈ **700 ms**

percentil 95 bajo carga alta (437 ms de media)
(≈ 3 ms en carga baja y media)

Sí, el clúster es viable para desplegar aplicaciones web de bajo costo: alcanzó ese nivel de **throughput** y **latencia** atendiendo **toda la carga sin un solo fallo**. Su límite está en la **red**, no en el cómputo ni en la temperatura.

Conclusiones — Cumplimiento de los objetivos

Objetivo general — cumplido

- ▶ Quedó **medido el nivel de latencia y throughput** del clúster (21 ejecuciones consistentes) y, además, el estudio **determinó el factor que lo condiciona**.

Objetivo específico 1 — cumplido

- ▶ El **prototipo físico resultó estable, funcional y reproducible**: operó sin un solo fallo en las 21 ejecuciones, quedando listo para la orquestación.

Objetivo específico 2 — cumplido

- ▶ El **despliegue web con Kubernetes fue exitoso**, con las métricas registradas bajo carga, y el análisis **explicó qué limita el rendimiento**.

Recomendaciones técnicas

- ▶ **Mejorar la red (lo más importante):** reemplazar los trabajadores por placas con conexión de red propia (Raspberry Pi 4 o superior) y usar un conmutador de red más rápido. Cambiar solo las placas no bastaría, porque la red seguiría limitada.
- ▶ **Mantener los ventiladores:** no por el rendimiento web, sino para proteger los equipos —bajar 10 a 15 °C puede duplicar su vida útil— y dejar el clúster listo para tareas más exigentes.
- ▶ **Almacenamiento:** preferir discos de estado sólido en lugar de tarjetas de memoria, para evitar su desgaste con el uso prolongado.

Recomendaciones para trabajos futuros

- ▶ **Repetir el estudio** con un clúster de placas iguales y red dedicada, para confirmar de forma directa que el límite está en la red.
- ▶ **Probar otros tipos de carga** (bases de datos, inteligencia artificial, archivos grandes) para ver si el factor que limita cambia y si la temperatura pasa a importar.
- ▶ **Hacer más pruebas y más largas**, para permitir un análisis estadístico y observar el desgaste a largo plazo.
- ▶ **Uso educativo:** aprovechar el prototipo en materias de redes, sistemas distribuidos e infraestructura, por su bajo costo.

Gracias por su atención.

Diego Oswaldo Márquez Paccha — Universidad Nacional de Loja